

Prompt:

Objective

Write a C program that picks a secret number between 1 and 10. The user gets up to 3 tries to guess the number. Requirements

- Use a fixed secret number (for example, 7).
- Use printf to prompt the user and scanf to read their guess.
- Use a while or for loop to let the user guess up to 3 times.
- Use if-else statements to check if the guess is too high, too low, or correct.
- Stop the loop immediately if the user guesses the correct number.
- Print a final win or lose message.

Example Output

text

Guess a number between 1 and 10.

Attempt 1/3. Enter your guess: 5

Too low! Try again.

Attempt 2/3. Enter your guess: 8

Too high! Try again.

Attempt 3/3. Enter your guess: 7

Correct! You win!

Co-pilot (accessed through website):

```
#include <stdio.h>
```

```
int main() {  
    int secret = 7;    // fixed secret number  
    int guess;  
    int attempt;  
  
    printf("Guess a number between 1 and 10.\n");  
  
    for (attempt = 1; attempt <= 3; attempt++) {  
        printf("Attempt %d/3. Enter your guess: ", attempt);  
        scanf("%d", &guess);
```

```

    if (guess == secret) {
        printf("Correct! You win!\n");
        return 0; // stop immediately
    } else if (guess < secret) {
        printf("Too low! Try again.\n");
    } else {
        printf("Too high! Try again.\n");
    }
}

// If loop ends without a correct guess
printf("Sorry, you lose! The number was %d.\n", secret);

return 0;
}

```

Test execution: Run the code to verify it actually works.

- Yes, it handles sample input.

Edge cases: Check if the code handles unusual inputs, empty values, or boundary conditions rather than just the happy path.

- No, it doesn't handle a string input or any numbers outside of the range 1-10.

Error handling: Look for meaningful error messages instead of silent failures or crashing.

- Crashes if you enter a string

Time Complexity:

- uses a For loop which is the most efficient option

Space Complexity:

- used 3 constants

Maintainability:

- Limited comments.

Claude ai (accessed through website):

```
#include <stdio.h>
```

```
int main() {
```

```
    int secret = 7;
```

```
    int guess;
```

```

int attempts = 3;

int won = 0;

printf("Guess a number between 1 and 10.\n");

for (int i = 1; i <= attempts; i++) {
    printf("Attempt %d/%d. Enter your guess: ", i, attempts);
    scanf("%d", &guess);

    if (guess == secret) {
        printf("Correct! You win!\n");
        won = 1;
        break;
    } else if (guess < secret) {
        printf("Too low! Try again.\n");
    } else {
        printf("Too high! Try again.\n");
    }
}

if (!won) {
    printf("Out of attempts! The number was %d. You lose.\n", secret);
}

return 0;
}

```

Test execution: Run the code to verify it actually works.

- Yes, it handles sample input.

Edge cases: Check if the code handles unusual inputs, empty values, or boundary conditions rather than just the happy path.

- No, it doesn't handle a string input or any numbers outside of the range 1-10.

Error handling: Look for meaningful error messages instead of silent failures or crashing.

- Crashes if you enter a string

Time Complexity:

- uses a For loop which is the most efficient option

Space Complexity:

- used 4 constants and uses an if statement after the for loop which is less efficient than either a for-else or simply a print afterword.

Maintainability:

- no comments.

Result:

I will go with the CoPilot code due to doing better or the same in every category, they both used a for loop, they both lacked error handling, CoPilot had less constants, and CoPilot had better maintainability.

Improvements:

I will add error handling for strings, a warning that tells you your guess was out of the provided bounds, and better maintainability through a prolog comment and comments on every line.